

Equipo docente: Mg. María Alejandra Vranic
Esp. Lic. Gustavo Siciliano
Esp. Lic. Ezequiel Scordamaglia
Lic. Oscar Ruina

Listas

Manejo de Listas	2
Librerías	2
Acceso a una lista	2
Métodos de búsqueda de primera aparición	2
Métodos de incorporación de elementos a la lista	3
Agregación	3
Composición	3
Modificación de elementos de una lista	4
Métodos de búsqueda de todas las ocurrencias	4
Formas de recorrer las listas en el test	5
Extra: Análisis de casteo	6
Práctica	7
5.A (Proyecto Incaa - NIVEL 1)	7
5.B (Proyecto Incaa - NIVEL 2)	8
5.C (Proyecto Supermercado - NIVEL 1)	8
5.D (Proyecto Supermercado - NIVEL 2)	9
5.E (Proyecto Supermercado - NIVEL 3)	9

Manejo de Listas

En la vida cotidiana todos usamos listas: cuando hacemos las compras, cuando preparamos un viaje o cuando pensamos en las cosas que debemos llevar a un campamento. Vamos anotando los elementos a medida que nos acordamos y, si es necesario, podemos ordenarlos o intercalar uno en el medio.

En Java, las listas cumplen la misma función: nos permiten guardar un conjunto de elementos y trabajar con ellos de manera dinámica (agregando, buscando, eliminando o recorriendo).

Una lista en Java debe declararse de la siguiente manera: **List<String> nombreLista;** y luego debe inicializarse, por ejemplo, con un ArrayList: **nombreLista = new ArrayList<String>();**

Nota: Declaramos la lista con la interfaz (List<String>) para mantener flexibilidad, y la inicializamos con una implementación (new ArrayList<String>()) que es la que realmente construye el objeto en memoria. En las próximas guías veremos estos conceptos con más detalle.

Tanto las listas como los arreglos (denominados colecciones de manera general) pueden contener en sus elementos tipos de datos primitivos o de objetos.

Librerías

Para utilizar listas en Java es necesario importar dos librerías:

```
import java.util.List;
import java.util.ArrayList;
```

Estas librerías se deberán incorporar en todas las clases que se utilicen Listas, ya sea en clases del modelo o clases de test.

Acceso a una lista

Una vez que tengamos inicializada una lista como **ArrayList**, hay muchos métodos que podemos utilizar para trabajar sobre los elementos. Algunos de los métodos más utilizados son:

- Agregar un elemento al final de la lista con **add()**.
- Consultar el tamaño de la lista con **size()**.
- Acceder a un elemento por su índice con **get(índice)**.
- Eliminar un objeto por su índice con **remove(índice)**.
- Verificar si está vacía con **isEmpty()**.

Métodos de búsqueda de primera aparición

En muchos casos necesitamos buscar un elemento dentro de una lista según un criterio específico. Por ejemplo, localizar un Usuario a partir de su DNI. Si la lista es un atributo de una clase, podemos definir en esa misma clase un método del tipo **traer**. Este método intentará localizar el objeto dentro de la lista y:

- retornará el objeto encontrado si cumple con el criterio de búsqueda
- o retornará null en caso de no hallarlo.

Nota: Un punto importante en este tipo de métodos es el uso de un while con doble condición de corte: Buscamos mientras no hayamos encontrado lo que queremos y, al mismo tiempo, nos aseguramos de no pasarnos del tamaño de la lista. De esta manera, el recorrido se corta en cuanto se cumple una de las dos condiciones, evitando trabajo innecesario.

Métodos de incorporación de elementos a la lista

Además de usar directamente **add()**, podemos crear métodos propios que incorporen un nuevo objeto a la lista, aplicando cierta lógica extra. Esa lógica puede incluir:

- Transformaciones en los datos, como calcular el próximo id automáticamente.
- Validaciones de atributos, que verifican que la clase permita asignar dichos valores (para que estas validaciones funcionen se colocan en el set de cada atributo de la clase y se utiliza el set en el Constructor)
- Validaciones, por ejemplo, comprobar que no exista ya un elemento con el mismo valor de algún atributo.

Nota: La forma en que se disparan Excepciones cuando no se cumple alguna validación se verá más adelante.

En cuanto al diseño, existen dos formas principales de agregar elementos:

Agregación

- El método recibe por parámetro un objeto ya instanciado.
- Ese objeto fue creado previamente fuera de la lista.
- Si la lista desaparece, el objeto sigue existiendo, porque no depende de ella.

Ejemplo:

```
public boolean agregarUsuario(Usuario usuario) {  
    return lstUsuarios.add(usuario);  
}
```

Composición

- El método recibe por parámetro los datos (atributos sueltos).
- Dentro del método se instancia el nuevo objeto y se lo agrega a la lista.
- En este caso, los objetos “nacen” dentro de la lista y dependen de ella: si la lista se elimina, también se eliminan los objetos que contenía.

Ejemplo:

```
public boolean agregarUsuario(String nombre, String apellido, long dni) {  
  
    //Transformaciones o cálculos antes de crear el objeto  
  
    Usuario nuevo = new Usuario(nombre, apellido, dni);  
    return lstUsuarios.add(nuevo);  
}
```

La segunda forma es la que utilizaremos con más frecuencia, ya que nos permite realizar validaciones de atributos que se ejecutan en el Constructor y transformar los datos antes de crear el objeto.

En el caso de que la clase Usuario tenga un atributo idUsuario, en el método **agregarUsuario** no se recibe el **idUsuario** como parámetro, sino que lo calcula de forma auto-numérica.

La regla para calcular el próximo id es la siguiente:

- Si la lista está vacía, el primer objeto recibirá el id 1.
- Si la lista ya tiene elementos, se busca el último y se le suma 1 al dicho id.

Esto se hace porque la lista es dinámica: los elementos pueden agregarse o eliminarse en cualquier momento. Por lo tanto, no es seguro usar **size()** como id, ya que ese valor puede variar y no garantiza la continuidad de la numeración.

Tampoco es recomendable usar una variable global que mantenga el último id asignado, porque:

1. Esa variable podría desincronizarse si se eliminan elementos de la lista.
2. Se perdería su valor si la aplicación se reinicia.
3. Introduce un punto único de fallo: si alguien modifica accidentalmente la variable, toda la numeración quedaría inconsistente.

En cambio, calcular siempre el próximo id a partir del último elemento de la lista asegura que la numeración sea **consistente, secuencial y confiable**, sin depender de variables externas.

Modificación de elementos de una lista

Cuando trabajamos con listas en Java, lo que realmente se guarda dentro de la lista no son los objetos en sí mismos, sino referencias a los objetos en memoria. Esto significa que, si traemos un objeto desde la lista y lo guardamos en una variable, ambas (la variable y el elemento de la lista) están apuntando al mismo objeto. No se hace una copia independiente.

Por esa razón, si a ese objeto que trajimos le cambiamos un atributo mediante un setter, el cambio se refleja automáticamente en la lista. No es necesario volver a insertarlo ni reemplazarlo: la lista sigue apuntando al mismo objeto, pero ahora ese objeto tiene sus valores actualizados.

Métodos de búsqueda de todas las ocurrencias

Para obtener una lista de elementos que cumplan con cierta condición, como podría ser “devolver todos los usuarios que tengan más de 30 años” se podrían desarrollar métodos que devuelvan listas.

Ejemplo:

```
public List<Usuario> traerUsuariosMayoresA(int edadReferencia) {  
    List<Usuario> usuariosFiltrados = new ArrayList<Usuario>();  
  
    //Realizar búsqueda sobre la lstUsuarios y agregar cada usuario que cumple la condición a la lista  
    usuariosFiltrados  
  
    return usuariosFiltrados ;  
}
```

En este caso se construye una nueva lista con los resultados que cumplen la condición pedida, dejando intacta la lista original.

Nota: Recordar utilizar una estructura de repetición que permita recorrer todos los elementos de la lista.

Formas de recorrer las listas en el test

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Scanner;

public class TestListas {
    public static void main(String argv[]) {
        Scanner in =new Scanner( System.in);
        System.out.println("Vamos a llenar un ArrayList del tipo String, el contenido de c/elemento es un String ");

        //inicializamos un objeto lista de tipo String
        List<String> lista = new ArrayList<String>();

        //Entrada de las cadenas
        String elem="";
        while(! (elem.equalsIgnoreCase("stop"))){
            System.out.println("Para finalizar tipear stop");
            elem= in.next();
            if (! (elem.equals("stop")))
                lista.add(elem);
        }

        System.out.println("1) Impresión implementando foreach loop");
        for (String s : lista) {
            System.out.println(s);
        }

        System.out.println("2) impresión implementando for loop with index");
        for (int p = 0; p < lista.size(); p++) {
            System.out.println(lista.get(p));
        }

        System.out.println("3) impresión implementando Iterator<tipo>");
        for (Iterator<String> iter = lista.iterator(); iter.hasNext();) {
            System.out.println(iter.next());
        }

        System.out.println("4) impresión automática utilizando el toString() del objeto");
        System.out.println(lista); //[holá, como, estas]

        in.close();
    }
}
```

Extra: Análisis de casteo

La cuestión ocurre al castear por ejemplo entre long y double, porque un long utiliza 63 bits para el valor y 1 para el signo, mientras que el double usa 52 para la mantisa y 1 para el signo. el resto es el exponente. Si el long que queremos castear ocupa más de 52 bits va a ocurrir pérdida de precisión, porque el double sólo puede almacenar 52 bits significativos. Puede (no vamos a tener error) almacenarse porque el double usa exponente, pero se perderán los dígitos menos significativos.

El int ocupa 32 bits en java, así que no debería haber problema al castearlo a double, ya que entra, el problema aparecería si lo queremos castear a float (usa 23 bits para la mantisa), por ejemplo.

Algunos sugieren verificar que la longitud en bits del número no supere la de la mantisa y levantar una excepción, si así ocurre. Si hacemos el casteo de un int a un float directamente, java pierde los dígitos menos significativos y hace la conversión sin decir nada.

De todos modos, la conversión como la estamos haciendo es ineficiente, ya que pasa el int a un string y después ese string a un double y nos nos evita el problema, ya que tampoco funciona:

```
int i = 2147483638; //(2^31-10)
float f = (float)i;

System.out.println(i);
System.out.println(f);
System.out.println((int)f);
System.out.println((int)Float.parseFloat(String.valueOf(i)));
```

Esto devuelve:
2147483638
2.14748365E9
2147483647
2147483647

En ambos casos se pierde precisión, y la conversión a String y float es más cara.

REALIZAR PRÁCTICA: 5.A (Proyecto Incaa - NIVEL 1)

REALIZAR PRÁCTICA: 5.B (Proyecto Incaa - NIVEL 2)

REALIZAR PRÁCTICA: 5.C (Proyecto Supermercado - NIVEL 1)

REALIZAR PRÁCTICA: 5.D (Proyecto Supermercado - NIVEL 2)

REALIZAR PRÁCTICA: 5.E (Proyecto Supermercado - NIVEL 3)

Práctica

5.A (Proyecto Incaa - NIVEL 1)

Modelo:

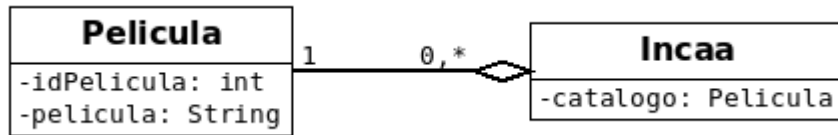


Diagrama de Clases

La clase Pelicula, con los atributos idPelicula tipo int, pelicula tipos String.

Por último la clase Incaa con el atributo catalogo del tipo List<Pelicula> donde se van a realizar Altas, Modificaciones y Bajas de objetos. Por lo tanto existe una relación de agregación entre Incaa y Pelicula de uno a muchos, la objeto Incaa va tener muchos objetos Pelicula en la lista.

Casos de uso:

+ agregarPelicula (String pelicula) : boolean

Si la película existe en la lista lanzar la excepción (Dejar para más adelante)

+ traerPelicula (int idPelicula) : Pelicula

Si la película no existe devolver null

+ traerPelicula (String partePelicula) : List<Pelicula> //todas las películas que contengan en su título el string partePelicula

+ modificarPelicula (int idPelicula, String pelicula) :

Modificar la película traerPelicula por id, si no existe la película lanzar la excepción (Dejar para más adelante), de lo contrario setPelicula con el parámetro pelicula .

+ eliminarPelicula (int idPelicula): boolean

Eliminar la película: traerPelicula por su id, si no existe la película lanzar la excepción (Dejar para más adelante), de lo contrario eliminar el elemento de la lista (remove)

Test: crear los test de para cada caso de uso con sus escenarios.

El paradigma de Objetos se basa en la reutilización de los Casos de Uso. En el momento de implementar cada comportamiento, es bueno detenerse a pensar si es posible reutilizar otro Caso de Uso ya implementado; con el fin de que la funcionalidad del sistema exista en un solo método, lo que conduce a un mejor diseño y facilita el mantenimiento.

5.B (Proyecto Incaa - NIVEL 2)

Modelo:

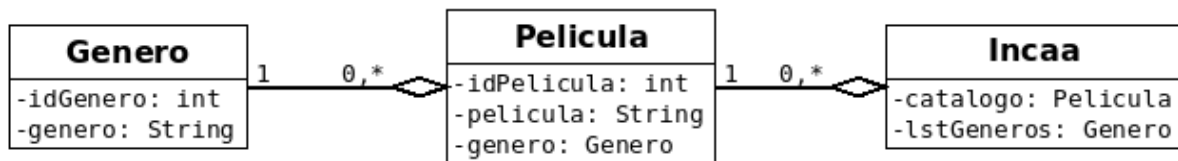


Diagrama de Clases

La clase Genero, con los atributos idGenero tipo int y genero tipo String.

La clase Pelicula, con los atributos idPelicula tipo int, pelicula tipos String y genero tipo Genero, por lo tanto existe una relación de composición entre las clases Pelicula y Genero, con cardinalidad de muchos a uno o sea que puede haber muchas películas de un género (no consideramos que una película pueda pertenecer a más un género para simplificar el problema).

Por último la clase Incaa con el atributo catálogo del tipo List<Pelicula> donde se van a realizar Altas, Modificaciones y Bajas de objetos.

Crear la clase Genero como indica el diagrama de clases y en Pelicula agregar el atributo genero.

Casos de uso:

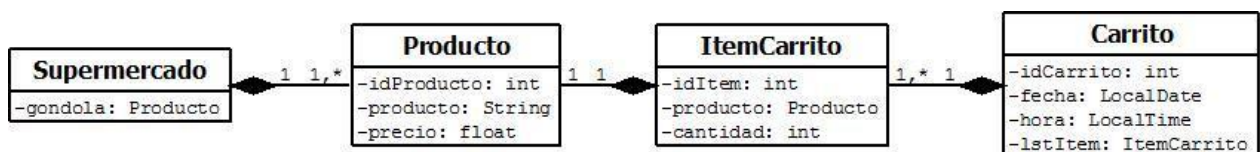
Sobrecargar el método traer película:

+ traerPelicula(Genero genero) : Pelicula

Test: Traer las películas del catálogo de un género e imprimirlas.

5.C (Proyecto Supermercado - NIVEL 1)

Modelo:



1) **+ agregarProducto(String producto, float precio) : boolean**

Si el producto existe en la lista lanzar la excepción (Dejar para más adelante)

2) + traerProducto(int idProducto) : Producto

Si el producto no existe devolver null

3) + modificarProducto(int idProducto, String producto, double precio) : boolean

Modificar el producto: traer producto por id, si no existe el objeto levantar la excepción (Dejar para más adelante), de lo contrario volver setear producto y precio.

4) + eliminarProducto(int idProducto) : boolean

Eliminar producto: traer producto por id, si no existe el objeto o existe en algún carrito levantar la excepción (Dejar para más adelante), de lo contrario eliminar el objeto (remove).

5) + agregarItem(Producto producto, int cantidad) : boolean

Cuando se agrega un producto al carrito si producto existe en algún item solo se incrementará la cantidad de lo contrario se agregará el item.

6) + eliminarItem (Producto producto, int cantidad) : boolean

En el caso de eliminar un producto del carrito si la cantidad es la misma se eliminará el item, si es menor se decrecerá la cantidad y de lo contrario, si no existe el ítem que contenga el producto lanzará una excepción (Dejar para más adelante).

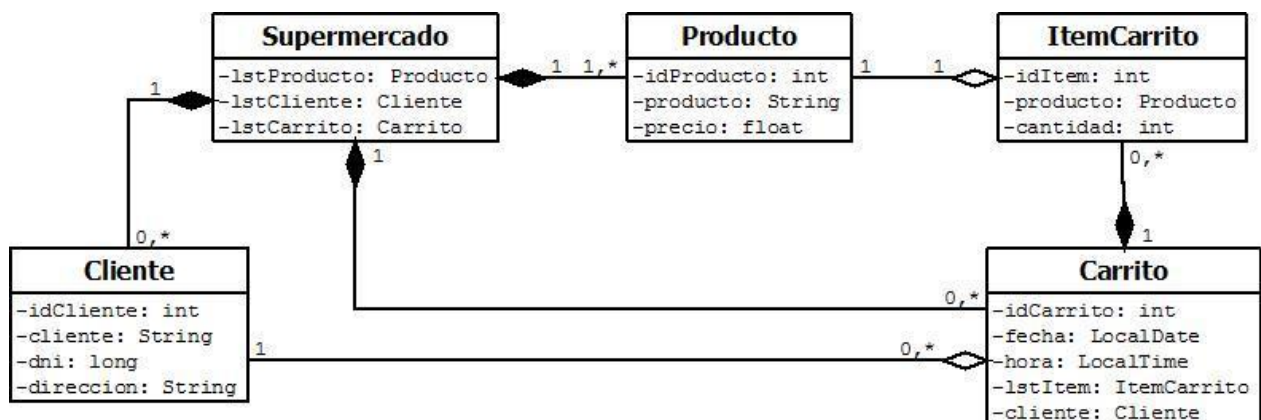
7) + calcularSubTotal() : float

8) + calcularTotal() : float

Test: crear los test de para cada caso de uso con sus escenarios.

5.D (Proyecto Supermercado - NIVEL 2)

Modelo:



Terminar de implementar las clases y los casos de uso del diagrama.

Casos de uso a implementar en las clases correspondientes:

- 1) + agregarProducto(String producto, float precio) : boolean
- 2) + traerProducto(int idProducto) : Producto
- 3) + eliminarProducto(int idProducto) : boolean
- 4) + agregarItem(Producto producto, int cantidad) : boolean
- 5) + eliminarItem (Producto producto, int cantidad) : boolean
- 6) + calcularSubTotal() : float
- 7) + calcularTotal() : float
- 8) +agregarCliente(String cliente, long dni, String direccion): boolean
Si el cliente existe en la lista levantar una excepción. (Dejar para más adelante)
- 9) +traerCliente(int idCliente): Cliente
Si el cliente no existe devolver null.
- 10) +eliminarCliente(int idCliente): boolean
Si el cliente no existe se debe levantar una excepción. (Dejar para más adelante)
- 11) +agregarCarrito(LocalDate fecha, LocalTime hora, cliente Cliente): boolean
Si el carrito existe en la lista levantar una excepción. (Dejar para más adelante)
- 12) +traerCarrito(int idCarrito): Carrito
Si el carrito no existe devolver null.
- 13) +eliminarCarrito(int idCarrito): boolean
Si el carrito no existe en la lista levantar una excepción. (Dejar para más adelante)
- 14) +calcularTotal(Cliente cliente): float
Si el cliente no existe levantar una excepción. (Dejar para más adelante)
- 15) +calcularTotal(int dniCliente): float
Si el cliente no existe levantar una excepción. (Dejar para más adelante)
- 16) +calcularTotal(LocalDate fechaInicio, LocalDate fechaFin): float
- 17) +calcularTotal(LocalDate fecha): float
- 18) +calcularTotal(int mes, int anio): float
Si el mes es incorrecto (por ejemplo 23), levantar una excepción. (Dejar para más adelante)
- 19) +calcularTotal(LocalDate fechaInicio, LocalDate fechaFin,
Cliente cliente): float
Si el cliente no existe levantar una excepción. (Dejar para más adelante)
- 20) +calcularTotal(LocalDate fecha, Cliente cliente): float
Si el cliente no existe levantar una excepción. (Dejar para más adelante)
- 21) +calcularTotal(int mes, int anio, Cliente cliente): float
Si el cliente no existe y/o el mes es incorrecto levantar una excepción. (Dejar para más adelante)
- 22) +calcularTotal(int mes, int anio, int dniCliente): float
Si el cliente no existe y/o el mes es incorrecto levantar una excepción. (Dejar para más adelante)

Test: crear los test de para cada caso de uso con sus escenarios.